Dynamic Page Policy Using Perceptron Learning

Muhammad M. Rafique
Department of Electrical and Computer Engineering,
University of Illinois at Chicago, Chicago, IL USA.
mrafiq2@uic.edu

Zhichun Zhu
Department of Electrical and Computer Engineering,
University of Illinois at Chicago, Chicago, IL USA.
zzhu@uic.edu

ABSTRACT

Modern multicore processors running data-intensive applications concurrently generate memory requests that exhibit different spatial and temporal locality. These interleaved memory requests result in an increased random memory access pattern as seen by the memory controller. Different page policies have been implemented to decide if a row should be kept open or close after an access. Appropriate choice of page policy is critical in reducing the memory access latency, especially in massively parallel main memory systems intended for high performance computing. In this paper, we introduce a dynamic page policy that uses perceptron learning to make the page open/closure decision by examining a wide variety of features extracted from application's memory access behavior. Simulation over multiprogramming SPEC CPU workloads indicates that our policy significantly reduces the memory access latency by reducing the row-buffer conflicts, and improves the overall performance by 5.5% on average as compared to a previously proposed dynamic page policy that makes adaptive page open/closure decision based on the bank-level page hit rate.

CCS CONCEPTS

• **Hardware**; • **Semiconductor memory;**; • **Computer systems organization**; • **Processors and memory architectures; Neural Networks;**;

KEYWORDS

Memory Systems, DRAM, High Bandwidth Memory, Perceptron Learning, Page Policy

ACM Reference Format:
Muhammad M. Rafique and Zhichun Zhu. 2022. Dynamic Page Policy Using Perceptron Learning. In *2022 International Syposium on Memory Systems (MEMSYS 2022), October 03–06, 2022, Washington, DC, USA.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3565053.3565055

1 INTRODUCTION

Increasing performance gap between the processor and main memory, commonly referred to as "Memory Wall" [1] has been a bottleneck in modern day computer systems. The off-chip memory bandwidth requirement by multi-core processors running data-intensive applications concurrently cannot be provided by traditional DDR-based main memory system due to limited bandwidth and scalability. To address this limitation, 3D-stacked DRAM based main memory system, such as High Bandwidth Memory [2] has been introduced. The 3D stacking of the DRAM chips over the top of a single logic layer provides substantial bandwidth and memory-level parallelism as hundreds of banks are available across multiple channels to serve memory access requests concurrently.

To maximize the utilization of available memory bandwidth, memory addresses are mapped across multiple banks, ranks, and channels that can be accessed in parallel. Since applications exhibit different spatial and temporal locality, smart page open/closure decision has to be made to reduce the row-buffer conflicts. Generally, for applications exhibiting higher locality, open-page policy is preferred in hope for the upcoming request to be served by the row currently opened in the row-buffer. On the contrary, close-page policy is preferred for the applications that exhibit lower locality as this policy takes precharge latency off the critical path to minimize overall latency.

Statically implemented open- or close-page policy cannot keep up with the dynamically changing memory access behavior as it cannot adapt to the changing locality in the memory access stream. To solve this problem, dynamic page policies have been introduced that help make page open/closure decision based on the application's run-time memory-access behavior. One class of dynamic page policies is 'access-based' policy [3], [4], [5] where a DRAM page is kept open in row-buffer for a certain number of accesses predicted by different prediction algorithms, and then closed afterwards. Another class is the 'time-based' policy [6], [7], [8] where a page is kept open for a certain time period and then closed afterwards. Dynamic page policies tend to perform better in making page open/closure decision compared to the statically implemented policies, as they are adaptable to the changing memory access behavior [6], [9].

To make an accurate page open/closure decision, a deep insight is required into the long-term application's memory access behavior. Previously proposed access-based techniques [3], [5] keep limited information about DRAM pages and their access behavior, which is later used to make the future page open/closure decisions. These policies may perform well in the short term where memory access patterns are repeated due to the intrinsic spatial and temporal locality found in the memory access stream, but do not perform very well in the long term due to lack of insight into the long-term application's memory access behavior.

In this paper, we introduce DYMPL, a DYnaMic page policy using an online Perceptron Learning model that captures the application's long-term memory access behavior and correlates different features to the page open/closure decision. Our proposed model learns from the past inaccurate page open/closure decisions to improve its

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MEMSYS 2022, October 03–06, 2022, Washington, DC, USA
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9800-8/22/10…$15.00
https://doi.org/10.1145/3565053.3565055

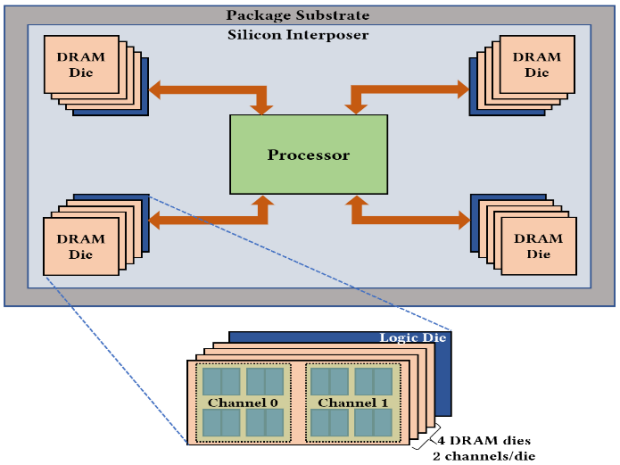

Figure 1: HBM Architecture

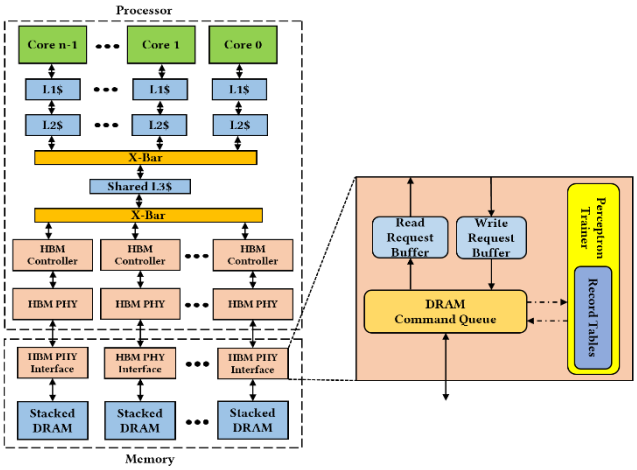

Figure 2: Perceptron Trainer in HBM Memory System

accuracy, and is adaptable to the changing application behavior where memory banks are accessed differently due to changing locality in the memory access stream.

Simulation results using multiprogramming SPEC CPU workloads [10], [11], [12] indicate that our perceptron-learning-based page policy reduces the row-buffer conflicts considerably, which results in a reduction in memory access latency and improves overall system's performance. Our scheme achieves a performance improvement of 5.5% on average, compared to a previously proposed dynamic page policy [9] that makes page open/closure decision for different banks based on the bank-level page hit rate. Compared with the statically implemented open- and close-page policies, our scheme can improve performance by 12.6% and 19.8% on average, respectively.

The rest of the paper is organized as follows: Section 2 overviews the High Bandwidth Memory architecture, dynamically changing application's memory access behavior, and basics of perceptron learning. Section 3 explains our proposed perceptron-learning-based page policy in detail. Experimental setup and details of different page policies used in our experiments are presented in section 4. Simulation results, analyses, and sensitivity studies on different parameters used in our experiments are presented in section 5. Section 6 discusses some of the related work and section 7 draws the conclusion of this paper.

2 BACKGROUND AND MOTIVATION

2.1 High Bandwidth Memory

shows general architecture of High Bandwidth Memory (HBM). Internally, it consists of multiple DRAM layers that are stacked on the top of a single logic layer. These DRAM layers are connected to logic layer via multiple high-speed Through Silicon Visas (TSVs) [13]. HBM exhibits immense memory-level parallelism and supports multiple memory channels, managed by their corresponding controllers. These memory channels are operationally and functionally independent to each other where memory management policies can be tailored to their specific access patterns in order to reduce the overall access latency and improve performance. Hundreds of banks spread across multiple channels help serve multiple memory transactions simultaneously, resulting in shallow read and write queues and improved memory response time.

The benefits provided by this memory organization makes it ideal for high-performance and bandwidth-sensitive systems like high-end desktops and servers. HBM-based system has 3D-stacked memory and processor laid out on a single interposer, as shown in Figure 1. Memory controller resides on the processor chip and is connected to the 3D-stacked DRAM module through wide interface. Logic area in the base of these 3D-stacked DRAM is of prime importance and has been proposed to integrate custom logic in order to enhance the functional and operational capabilities of the memory controller [9], [14], and to support processing-in-memory (PIM) architecture [15], [16]. In our proposed scheme, perceptron-learning-based page policy can be implemented in the logic base of HBM to improve performance (Figure 2).

2.2 Dynamic Memory Access Behavior

Modern data-intensive applications running concurrently on multiple processing cores exhibit dynamic memory access behavior, essentially seen as a random access stream by the memory subsystem. This behavior imposes different memory bandwidth requirements at different times, which ignites the competition on the shared resources like last-level cache and memory buses. Applications having larger data sets tend to access main memory more often since the whole data cannot fit into processor caches. Similarly, applications exhibiting lower temporal and spatial locality also tend to access main memory more often due to increased cache misses.

Memory access requests should be managed in such a way to ensure minimum memory access latency for all the application threads. Previous studies [17], [18] found that different applications running concurrently yield different cache miss rates and row-buffer hit rates. Even within a single application running on a single processing core, memory access behavior is not consistent throughout the whole application execution phase. Study in [9] shows varying row-buffer hit rate during different execution phases

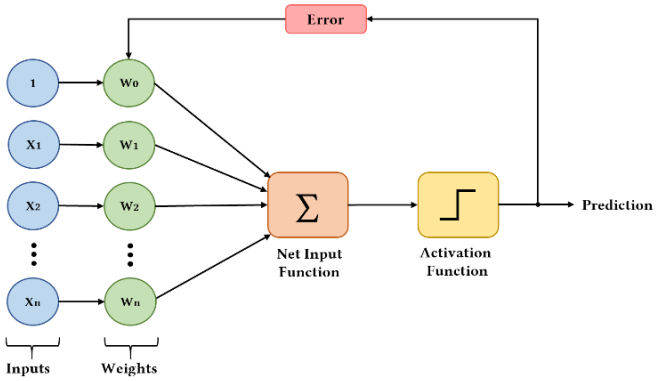

Figure 3: Perceptron Learning

Table 1: Different Page Mode Latencies

Open Page	Page Hit	tCL
	Page Conflict	tRP + tRCD + tCL
Close Page	Page Miss	tRCD + tCL

of SPEC CPU 2006 benchmarks running on a single processing core. Therefore, a statically implemented page policy is not ideal throughout the application execution and a deep insight into the long-term application behavior is required to adjust the page policy dynamically in response to the changing memory access behavior.

2.3 Perceptron Learning

Perceptron learning is an algorithm for supervised learning for binary classifiers [19]. As an abstract idea, perceptron learning uses a set of features and assigns a certain weight to each feature. Each feature is multiplied by its corresponding weight and a final weighted sum is calculated. This weighted sum is then passed through a threshold function called 'activation function'. If the output of the activation function exceeds a predefined threshold, a positive prediction is made, otherwise, a negative prediction is made. The prediction is then compared to the actual outcome of the event. If the prediction is wrong or the magnitude of the sum failed to reach a certain threshold, the weights are updated at a certain learning rate to train the perceptron. For the correct prediction, weights are not updated. This process is repeated until the whole process is converged, and the final weight values are settled down. Figure 3 shows the perceptron learning process.

From microarchitectural perspective, perceptron learning has been applied in branch prediction [20], prefetching [21], [22] and cache reuse prediction [23], [24]. In this paper, we apply perceptron learning to predict the page open/closure decision dynamically by learning memory access pattern and training the perceptron accordingly.

2.4 Motivation

The traditional DDRx-based main memory system can hardly meet the immense memory bandwidth requirements set by modern data-intensive applications running concurrently on multiple processing cores. Limited number of memory channels and banks are also a limiting factor in scaling up the bandwidth provided by current memory systems. High bandwidth memory (HBM) based memory system provides revolutionary improvements over DDRx-based main memory systems, including 3D stacking of DRAM chips that enables more memory capacity per unit area, high number of channels and banks that provides immense memory-level parallelism, and logic area in the base of HBM module that can be used to design custom circuits to complement the operational and functional capabilities of memory controller residing on the processor chip.

The presence of such large number of independent memory banks requires enhanced management policies in order to reach their full potential. The page policy that decides whether the current DRAM page in the row-buffer should be closed or remain open, is critical in determining the overall memory access latency. Table I shows the latencies for a page hit or a page conflict under statically implemented open-page mode or a page miss under statically implemented close-page mode. Here tRCD represents the latency to activate the row, tCL represents the latency to select a certain column from the activated row in the row-buffer, and tRP represents the latency to precharge the currently open row. For open-page mode, the worst-case latency is incurred in case of a row-buffer conflict, in which, the precharge latency is on the critical path and causes substantial delays when a memory request has to be served from a different row in the same bank. On the other hand, closing the row after each memory access in close-page mode can cause some of the requests becoming row-buffer misses that could have been hits since the same page has to be activated again to serve the request. This implies that statically implemented open- or close-page policy cannot work well for the whole application execution phase and a dynamic approach should be adopted to tackle this problem.

Traditional dynamic access-based and time-based page policies have their own limitations as they make their page open/closure decisions based on a limited history information about previous memory access behavior. To make more accurate page open/closure decision, application's long-term memory access behavior also need to be captured without keeping the exhaustive list of each page that has been accessed in the past. A scheme is needed to make run-time page open/closure decision that is adaptive to the changing application's behavior.

3 PERCEPTRON-BASED PAGE POLICY

We propose a dynamic page policy based on the perceptron learning algorithm. The main idea is to capture the memory access behavior by profiling the history of accesses, and page hit/miss behavior for all those accesses. Certain features are extracted from each memory access request to profile application's long-term memory access behavior. An online perceptron learning model is then used to predict the page open-closure decision based on the learned memory access behavior.

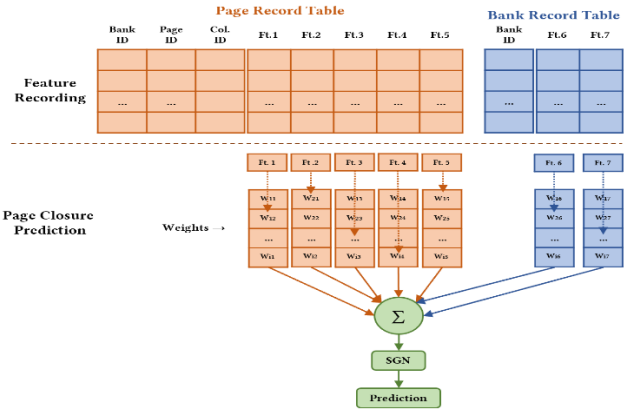

Figure 4: Feature Recording and Perceptron Prediction

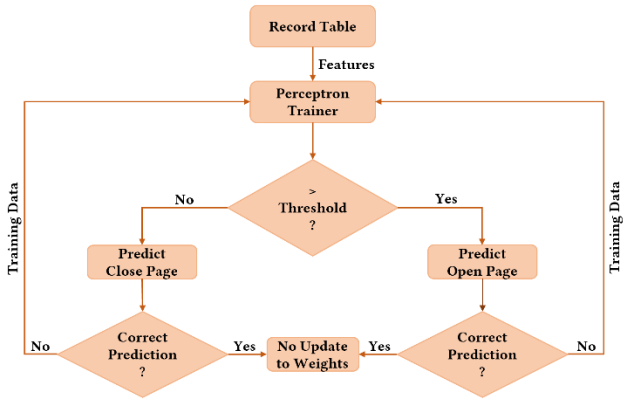

Figure 5: Perceptron Training

3.1 Perceptron Algorithm

Our perceptron-learning-based page policy uses certain features to comprehend the memory access behavior at the page-level and bank-level. Each page-level or bank-level feature is stored as a saturated-counter value in the separate tables called 'Page Record Table' (PRT) and 'Bank Record Table' (BRT), respectively. Corresponding to each feature, a separate 'Weight Table' (WT) is used to store the weights, which is a direct-mapped structure indexed by the counter value of the feature. Note that for a perceptron algorithm with N features, there are N Weight Tables, and each may have different number of entries. Each weight is 4-bit, ranging from -8 to +7. As shown in Figure 4, in order to reduce the computational overhead, our scheme implements a slightly modified version of the perceptron learning algorithm. Instead of using the costly multiplication operation, different counter values of the recorded features are used to index into the weight tables and corresponding weights are then summed up, similar to approach used in [21]. If the sum is greater than a threshold, the page is predicted to be kept open, otherwise, close page is predicted. The predicted page open/closure decision is then compared to the actual page hit or conflict in the row-buffer. If the prediction is correct, no further weight update is required. If the prediction is incorrect, training occurs, and the corresponding value of features are used to re-index into the weight tables and weights are updated accordingly.

3.2 Feature Selection

Since the perceptron accumulates the contribution from different features for prediction, the perceptron learning algorithm is critically dependent on the set of features used to train it. This makes the perceptron learning highly scalable as additional features can be added when needed, provided there is enough storage space available to store the selected features and timing and energy budget allows additional mathematical computation required to incorporate the impact of these added features.

For our proposed scheme, the following features have been selected that correlates the page open/closure decision to the program's memory access behavior. These features are derived from the memory access stream and are recorded in the record tables to train the perceptron as shown in Figure 4.

Page Utilization: This feature provides insight to the spatial locality found within the application. Once a DRAM page is opened in the row-buffer, the number of column hits are profiled before the page is closed. A higher utilization implies more locality found within an application, suggesting open-page policy. On the contrary, a lower page utilization implies a close-page policy where a bank is precharged when no pending requests go to the currently open page to keep the row-buffer ready for an upcoming request targeting a different DRAM page.

Page Hotness: This feature provides information about how often a certain DRAM page is accessed to serve the memory request. If a page is accessed more often, there is a high locality found in the application's memory access stream, implying open-page policy. Otherwise, if a page is not accessed very often, then the application is exhibiting less locality and prefers close-page policy. To keep track of the hotness, a counter is incremented every time a page is accessed while its entry is found in the Page Record Table.

Page Recency: This feature keeps track of the recency of each page accessed in a particular bank. If a page is accessed recently, there is a high chance that the page would be accessed again due to temporal locality, implying open-page policy. If a page is not accessed recently, close-page policy should be implemented to take the precharge latency off the critical path for the upcoming access. A counter is used to track the recency of each page, which is set to maximum when a page is accessed and the recency counters of all other pages within the same bank are decremented by one. Note that the Page Record Table has a set-associative structure, which is indexed by the bank ID in our experiment. So, it is easy to locate other pages in the bank.

Column Stride: This feature provides the information about the stride between consecutive column accesses in an open DRAM page. A larger column stride means there is an increasing chance that the next access is not a page hit in this particular bank and may result in a page conflict, favoring the close-page policy. On the contrary, a smaller stride increases the chance of page hit, implying open-page policy.

Page Hit Count: This feature keeps track of the number of hits and misses to a page. A saturating counter is used that increments

Table 2: SPEC CPU (2006 + 2017) Workloads

ID	Workload Sets
HM1	bwaves, lbm, fotonik3d, mcf, soplex, lbm, leslie3d, libquantum
HM2	mcf, omnetpp, bwaves, GemsFDTD, sphinx, soplex, milc, libquantum
HM3	cpugcc, xz, deepsjeng, cpuxalan, lbm, milc, libquantum, GemsFDTD
HM4	bwaves, xz, fotonik3d, cpuxalan, lbm, leslie3d, milc, soplex
LM1	perlbench, cactusBSSN, deepsjeng, leela, blender, imagick, exch.2, cpuxalan
LM2	deepsjeng, leela, sjeng, astar, bzip2, cactus, h264ref, perlbench
LM3	perlbench, leela, deepsjeng, blender, bzip2, h264ref, zeusmp, hmmer
LM4	xz, imagick, cpuxalan, blender, zeusmp, h264ref, astar, perlbench
MX1	leela, lbm, blender, fotonik3d, mcf, omnetpp, bwaves, GemsFDTD
MX2	bwaves, lbm, fotonik3d, mcf, tonto, gromacs, hmmer, gobmk
MX3	cpugcc, bwaves, mcf, deepsjeng, namd, astar, gamess, povray
MX4	nab, cpuxalan, leela, xz, libquantum, milc, omnetpp, GemsFDTD

Table 3: Baseline System Configuration

Parameter	Configurations
Processor	8 out-of-order cores @ 3.6 GHz, 8-issue superscalar, x86-64 ISA, 192 entry ROB.
Caches	L1 (I/D): 32KB (P), 4 way, 64B line, thit = 4 cycles,16 MSHRs L2: 256KB (P), 8 way, 64B line, thit = 10 cycles, 16 MSHRs L3: 16MB (S), 16 way, 64B line, thit = 20 cycles, 64 MSHRs Multi-banked, Inclusive, MOESI cache coherence protocol
HBM Specs.	8 GB, 8 channels, 8 DRAM layers, 16 banks/channel, 128 bit memory bus.
DRAM Specs.	500 MHz for 1Gbps DDR rate, Queue Size (R/W) = 32/64 1KB row-buffer size, 16384 rows/bank trcd = 15 cycles, trp = 15 cycles, tcl = 15 cycles.
Memory Controller	Memory Scheduling: First-Come First-Serve Address Mapping: Row:Rank:Bank:Channel:Column Page mode: Close Page

when there is a hit to this page and decrements when there is a miss. This feature informs about dynamically changing locality found within the application, which may change over different phases. Higher hit count implies open-page mode, while lower hit count implies close-page mode.

Bank Recency: Just like the page recency, bank recency provides information about how recently a bank is accessed. So, if a bank is recently accessed, there is a high probability that the access is going to the same page in a bank, which implies an open-page policy. On the contrary, if a bank is not recently accessed, close-page policy should be implied because there are no pending requests intended for this bank and the page should be precharged to give space for the upcoming request in the row buffer, and a bank can be put into low power mode to conserve energy. Just like page recency, bank recency is tracked by the recency counters. Every time a bank is accessed, it's recency counter is set to max, and all the recency counters of other banks within the channel are decremented by one.

Bank Hit Count: Just like page hit count, this feature keeps track of the number of hits and misses to a bank. A saturating counter is used that increments when there is a hit to this bank and decrements when there is a miss. Higher hit count implies open-page mode, while lower hit count implies close-page mode.

3.3 Perceptron Model for Page Open/Closure Prediction

3.3.1 Metadata Recording: Every time a DRAM page is accessed in a certain bank, both Page Record Table (PRT) and Bank Record Table (BRT) are populated (or updated). PRT is organized as a 32-way set-associative table consisting of sixteen sets. BRT is a direct-mapped structure, and each bank has one entry in the BRT, which is updated every time a certain bank is accessed. Both tables are indexed by the bank id of the requests (sensitivity studies of different PRT sizes are presented in section 5). Each entry in PRT consist of a bank id, page id, column id, page utilization, page recency, page hotness, column stride, and page hit count. BRT consist of bank recency, and bank hit-count. Figure 4 shows the feature recording and perceptron prediction procedure. The actual outcome of the page hit/miss is also recorded for the comparison to the predicted page mode in PRT. This metadata is collected for training purposes where we

Table 4: Storage Overhead of Different Record Table Sizes

Record Table Entries	256	512	1024	2048
Storage Overhead	1.37 KB	2.77 KB	5.65 KB	11.52 KB

Table 5: Metadata Stored in Bank Record Table

	Details	Bits	Comments
Physical Address	Bank ID	4	Bank ID of the request
Bank-Level Features	Bank Recency	4	Keeps track of Bank Recency
	Bank Hit Count	8	Tracks hits at bank-level

Table 6: Metadata Stored in Page Record Table

	Details	Bits	Comments
Physical Address	Bank ID	4 14	Bank ID of the request
	Row ID	4	Page ID of the request
	Column ID		Column ID of the request
Output	Page hit/miss	1	Bit to record page hit/miss status
Page-Level Features	Page Utilization	4	Keeps track of Page Utilization
	Page Hotness	4	Keeps track of Page Hotness
	Page Recency	5	Keeps track of Page Recency
	Column Stride	4	Tracks Column Stride per page
	Page Hit Count	4	Tracks hits at page-level

have to train the weights for each counter value of the features. Since our proposed model is online, learning continues throughout the application execution in order to adjust the prediction based on the dynamically changing application's memory access behavior.

3.3.2 Data Retrieval and Prediction: When a page open/closure decision is to be made after an access, the bank and page id of the currently opened page is searched in the record tables. If the entry is found, all the counter values of the corresponding features are used to index into the respective weight tables, and the corresponding weights are then summed up. If the sum is equal or greater than 0 (the threshold), the page is kept open. If the sum is smaller than 0, the page is closed. If the entry of page is not found in the PRT, a new entry is added, and the page is kept open by default. If the entry is not found, and the number of page entries are full corresponding to a bank, the record of least recently accessed page is evicted from PRT to make room for the record of the new page.

3.3.3 Training: If the page is kept open in the row-buffer based on the perceptron prediction, the scheme will check for the upcoming request to the same bank. If it is a row-buffer hit, the prediction was correct. In such case, no update to the weights is necessary. If there is a row-buffer conflict by the upcoming request, the prediction was wrong. The counter value of each feature that was used for the prediction is used to index the respective weight table and corresponding weights are decremented. Same procedure is applied in case of a close-page prediction, except the weights are incremented this time if the close-page prediction turns out to be wrong. No training of weights is required in case of a correct prediction, meaning an upcoming request targets a different row within the same bank. Figure 5 summarizes the flow of training process.

4 EXPERIMENTAL METHODOLOGY

4.1 Simulation Configuration

We evaluate our perceptron-based dynamic page policy using a cycle accurate x86-64 simulator gem5 [25]. The simulator is extended to support different page policies for comparison. The details of our baseline simulation configuration are shown in Table II. Our simulated HBM model is based on [2] and [13]. The energy consumption is estimated using model presented in [26], [27]. The detailed memory controller operations are based on the model presented in [28] whereas specifications are obtained from [29]. The address mapping scheme assigns lowest order bits to the column address to ensure that the sequential cache lines occupy the same DRAM row to maximize page hits.

4.2 Workloads

We evaluate our scheme using a mix of SPEC CPU2006 and SPEC CPU2017 Benchmarks [10], [11], [12]. We use 12 different sets of multiprogramming workloads, and each set is categorized based on misses-per-kilo-instructions (MPKI) values. These MPKI values are determined by running each benchmark on a single-core system with last level cache size of 512KB [30]. Each set contains a mix of eight different SPEC benchmarks, hence giving a wide range of workloads. Based on the range of MPKI values, the workload sets are categorized as follows:

High Memory Intensive (HM): MPKILLC > 20

Low Memory Intensive (LM): $1 \leq$ MPKILLC ≤ 20

Mixed Memory Intensive (MX): 4 (HM) + 4 (LM)

The details of each workload set are shown in Table III. For each SPEC benchmark, "reference" data set is used. The simulations are fast-forwarded through their initialization phase, caches are warmed-up for an additional 100 million instructions, and perceptron starts training during this phase. Then detailed simulations are run for 1 billion instructions for each set of workloads. The perceptron training is continued throughout the simulation of the workloads in order to capture the change in the application's memory access behavior and make corresponding change in the page policy.

4.3 Evaluated Schemes

In our experiments, we consider a variety of page policies that are implemented either statically (S) or dynamically (D):

CLOSE (S): the baseline close-page policy, where row-buffer is precharged if no pending request in the queue targets this row,

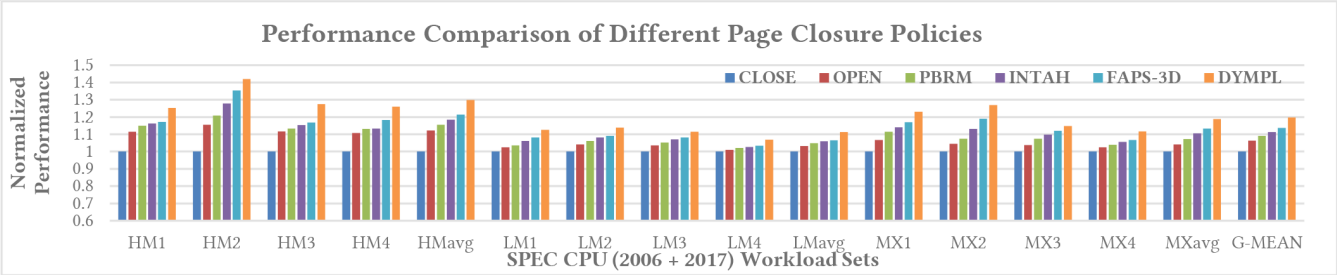

Figure 6: Performance Comparison of Different Page Policies

making room for the next row to be activated upon request. This scheme favors the applications with low spatial locality.

OPEN (S): the open-page policy, where row-buffer is kept open after the current access in anticipation of an upcoming request to be a hit in the open row. This scheme favors the applications with high spatial locality.

PBRM (D): Prediction Based Row-buffer Management [3]. This scheme uses a history table to keep track of the number of hits in recently opened rows before closing it. In this scheme, when a row is opened, a corresponding entry for that row is looked up in the table. If no entry is found, page is kept open until a conflict happens and the number of hits is recorded in the table before closing this row. If an entry is found, the row is closed after the estimated number of accesses as suggested by the table entry. The entries are updated periodically in the history table based on the correctness of the prediction.

Intel Adaptive HAPPY (INTAH) (D): HAPPY implementation of Intel Adaptive page closure policy [6], [30]. This is an implementation of time-based policy where a timing register (TR) and a timeout counter (TC) are used, and a page is kept open until TC reaches TR and then closed afterwards. A scheme is used to update the value of TR at runtime using a 4-bit mistake counter (MC) whose value is incremented when a page was wrongfully precharged that could have been a page hit, and decremented when a page was correctly prechraged that could have resulted in a page conflict. If MC is higher than a threshold, page is kept open for a longer period, otherwise page is closed sooner. This scheme was originally implemented in Intel Xeon X5650 [30] and authors in [6] implemented the modified scheme that achieved similar performance with noticeably lower overhead.

FAPS-3D (D): The feedback-directed, adaptive page management scheme [9], which is an implementation of dynamic access-based page policy proposed for 3D-stacked DRAM based main memory systems. This scheme monitors the bank-level row-buffer hit rate and classify the banks with low, high, and medium hit rates. A 2-bit saturating counter is kept per bank that is updated based on the bank-level page hit rate in a particular epoch and recommends open- and close-page policy for the next epoch for the banks identified as high-hit-rate and low-hit-rate banks, respectively.

Dynamic Page Policy Using Perceptron Learning (DYMPL) (D): Our proposed perceptron based page policy. Details of this policy have been presented in section 3.

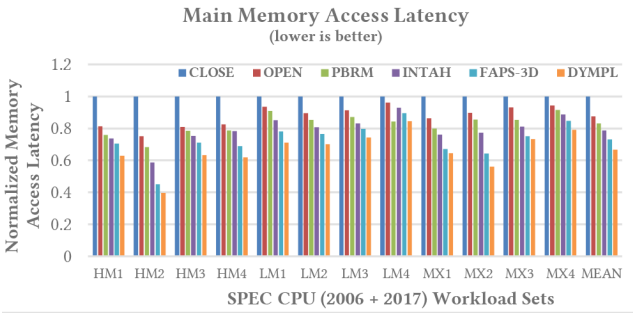

Figure 7: Main Memory Access Latency

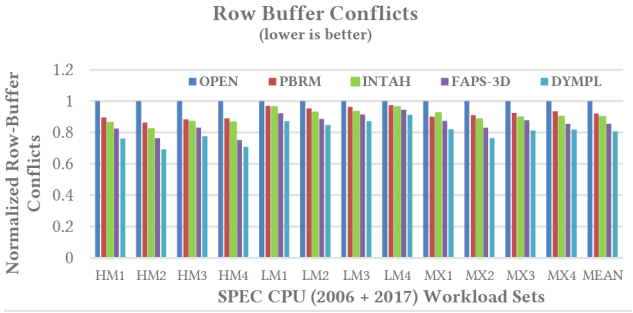

Figure 8: Row Buffer Conflicts

4.4 Overhead

Our perceptron-based adaptive page management scheme introduces some overhead. The record tables are kept per memory channel to store metadata used for perceptron training. The details of metadata stored for each entry is shown in Table IV and Table V. Based on the number of entries in the record tables used to train the perceptron, the overall hardware budget of storing metadata per memory channel is 2.77 KB for a PRT size of 512 entries, as shown in Table VI. The impact of having different number of entries in the record tables on performance is presented in the next section. In addition, separate perceptron Weight Tables are required corresponding to each feature to store the vector of weights. Each weight table has 2n entries where n is the number of bits used to store each feature. With each weight being 4-bit, this further requires a storage overhead of 184 Bytes per channel to store all the perceptron weights. Area and energy overhead for implementing SRAM-based

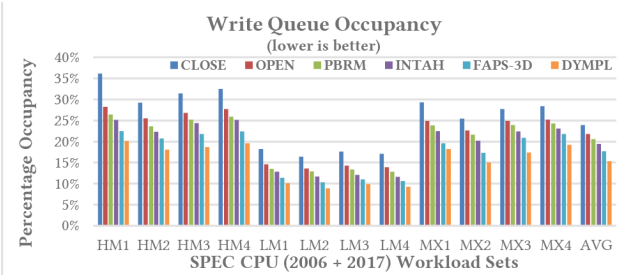

Figure 9: Write Queue Occupancy

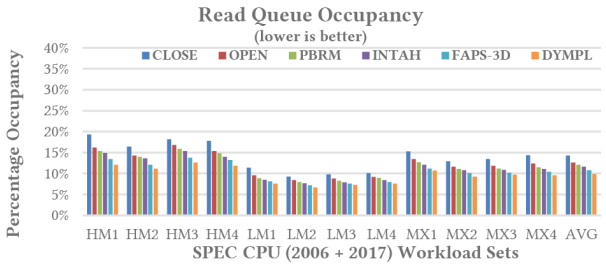

Figure 10: Read Queue Occupancy

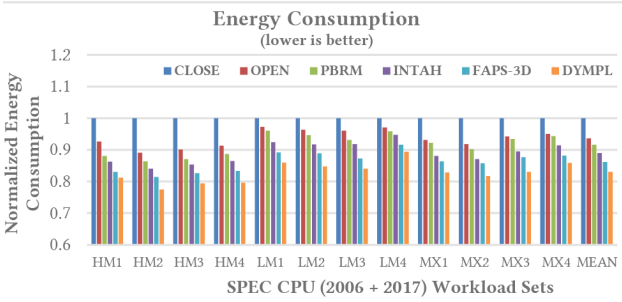

Figure 11: HBM Energy Consumption

storage using 32nm technology is estimated using CACTI 6.0 [31] and turned out to be ~0.08mm2 and ~0.022 nJ/access, respectively.

5 EVALUATION

5.1 Performance Analysis

Figure 6 shows the performance improvement obtained by our scheme over the CLOSE, OPEN, PBRM, INTAH, and FAPS-3D. The performance of each scheme is calculated by taking the geometric mean of relative IPC (instructions-per-cycle) of each application in the workload set, normalized to that of CLOSE. The average performance improvement of DYMPL for high-memory-intensive workloads (HMx) is 29.9% over CLOSE, OPEN achieves 12.2%, PBRM achieves 15.6%, INTAH achieves 18.4%, and FAPS-3D achieves 21.4% performance improvements compared to CLOSE, respectively. For low-memory-intensive workloads (LMx), DYMPL achieves a performance improvement of 11.2%, while OPEN, PBRM, INTAH, and FAPS-3D achieve performance improvements of 3.2%, 4.8%, 6.0%, and 6.6%, respectively. Finally, for the mixed-memory-intensive workloads (MXx), DYMPL achieves a performance improvement of 18.9%, while OPEN, PBRM, INTAH, and FAPS-3D achieve performance improvements of 4.1%, 7.2%, 10.5%, and 13.4%, respectively. Overall, our scheme DYMPL achieves an average performance improvement of 19.8% over CLOSE, while OPEN, PBRM, INTAH, and FAPS-3D achieve performance improvements of 6.4%, 9.0%, 11.3%, and 13.6%, respectively, across all 12 sets of workloads.

The performance gains are due to learning the long-term application's memory access behavior using perceptron learning and making page open/closure decision accordingly. The system also learns from the wrong predictions to improve upon itself and thus, achieves higher gains as compared to all other page policies that are implemented either statically, or dynamically based on the limited history information about memory access behavior. Note that high-memory-intensive workloads show highest performance gains as compared to the low- and mixed-memory-intensive workloads due to their highest dependence on the memory subsystem and thus, get most benefits from the improvements in main memory system.

5.2 Main Memory Access Latency

Figure 8 shows reduction in main memory access latency achieved by our perceptron-based page policy, which results in overall performance gain. The memory access latency shown here is the latency to access only the HBM-based main memory system and does not include the latency to access the caches. It includes the queuing latency, scheduling latency, and latency associated with the bank unavailability due to ongoing refresh operations. The results are normalized to the CLOSE and all other schemes are compared accordingly. The OPEN scheme achieves an average reduction of 12.4% over the CLOSE, PBRM achieves an average reduction of 16.9%, INTAH achieves an average reduction of 21.2%, FAPS-3D achieves a reduction of 26.9%, while our proposed scheme DYMPL achieves an average reduction in main memory access latency by 33.3% when compared to CLOSE. This latency reduction in accessing main memory is due to the adaptive open/closure of DRAM pages after an access in the row-buffer, by learning memory access behavior over time and making informed decision using perceptron learning about page open/closure. The accuracy of making page open/closure decision only gets better over time as our learning model adapts to the changing application behavior and makes well-informed predictions. One thing should be noted is that all the dynamic page policies perform better in reducing the main memory access latency as compared to static open- and close-page policies due to their ability to adapt to the dynamically changing memory access behavior.

5.3 Row-buffer Conflicts

The latency reduction in accessing main memory is a direct result of reduced row-buffer conflicts. Figure 7 shows the result of such reduction in row-buffer conflicts. The conflicts of OPEN policy are normalized to 100% and all other schemes are compared accordingly. The average reduction achieved in row-buffer conflicts is 7.9%, 10.1%, 14.5%, and 19.4% for PBRM, INTAH, FAPS-3D, and DYMPL, respectively, across all workload sets. CLOSE policy has been omitted from the comparison because most requests result in

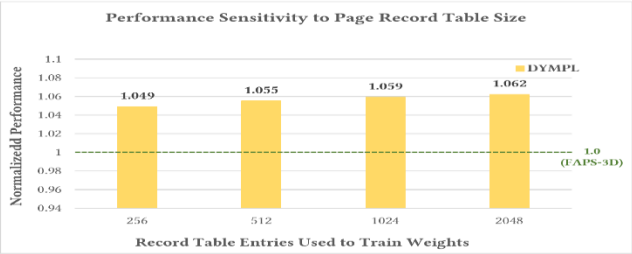

Figure 12: Performance Sensitivity to Different Page Record Table Sizes

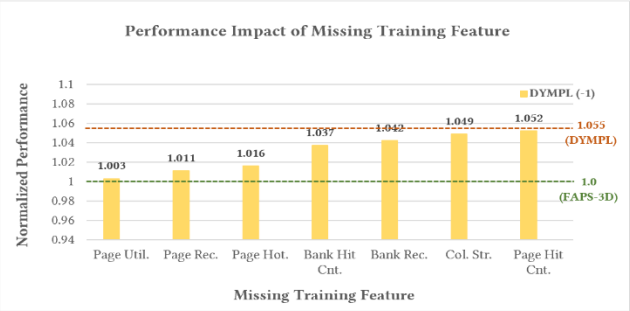

Figure 13: Performance Sensitivity to Missing Training Features

a miss when a page is closed due to no upcoming request to the same page.

As can be seen from Figure 7, dynamic page policies achieve considerable reduction in row-buffer conflicts since they do not keep the page open unnecessarily if there is not enough locality found within the application. This results in taking the precharge latency off the critical path for the requests targeting a different row within the same bank. As a result of comprehending long-term application behavior using perceptron learning, our scheme is able to apply open- and close-page policy dynamically with higher accuracy in order to reduce the row-buffer conflicts. High- memory-intensive workload sets get the largest reduction in row-buffer conflicts because of their memory intensive behavior, while low-memory-intensive workload sets get the lowest reduction due to lower memory access rate and lower row-buffer conflicts. Managing the page policy dynamically for each bank and closely monitoring the bank access behavior help in making smart page open/closure decisions, resulting in reduced row-buffer conflicts, lower memory access latency, and overall performance improvement.

5.4 Queue Occupancy

Due to the availability of high memory-level parallelism in High Bandwidth Memory made possible by hundreds of banks spread across multiple channels, the memory request queues are shallower as compared to DDRx-based main memory system. Figure 9 and Figure 10 show average occupancy for write and read queues, respectively, as seen by an incoming request. For read queues, DYMPL has an average occupancy of 9.9%, while CLOSE, OPEN, PBRM, INTAH, and FAPS-3D, have an average occupancy of 14.3%, 12.6%, 12.1%, 11.6%, and 10.8%, respectively.

Similarly, for write queues, DYMPL has an average occupancy of 15.3%, while CLOSE, OPEN, PBRM, INTAH, and FAP-3D have an average write queue occupancy of 23.9%, 21.8%, 20.6%, 19.4%, and 17.5%, respectively. This reduction is achieved due to fewer row-buffer conflicts, resulting in the efficient processing of requests and hence lower waiting time for each memory request to get processed. Since priority is given to the read requests, the write queue occupancy is relatively higher because write requests can be buffered in order to efficiently process the read requests.

5.5 Energy Consumption

The average energy consumption of HBM across all channels is shown in Figure 11. The average energy consumption of CLOSE is normalized to 100% and all other schemes are compared accordingly. Our scheme DYMPL reduces the overall memory energy consumption by 16.9% on average compared to CLOSE, while OPEN, PBRM, INTAH, FAPS-3D achieve an average reduction of 6.4%, 8.4%, 11.0%, and 13.8%, respectively. The detailed energy model used in our simulation incorporates energy consumed by activate/precharge operations, read and write operations, refresh operations, and energy associated with dynamically managing power-down mode when no references are due [26], [27].

The reduction in overall energy consumption is due to accumulated savings from fewer activation and precharge operations as compared to CLOSE, where same row may have to be activated again that it just precharged. OPEN policy accounts for less energy consumption due to fewer precharge and activate operations as a result of consecutive hits to the current open row. Similarly, the schemes PBRM, INTAH, and FAPS-3D consume less energy due to their better accuracy in predicting open- and close-page mode dynamically based on the changing locality found within the application. Since our proposed policy DYMPL improves the prediction accuracy of page open/closure decision by using perceptron learning, it achieves substantial reduction in row-buffer conflicts and hence, corresponding reduction in energy consumption. Moreover, additional energy is saved by putting the banks in low-power mode when no requests are due.

5.6 Sensitivity Analysis

5.6.1 Training Features: Perceptron learning uses different features to train the perceptron and each feature may have a high or low impact on the final prediction, as determined by its weight. To analyze the impact of each feature, we run the experiments by skipping each training feature one at a time, and observing its impact on the overall system performance. Figure 13 shows the performance impact of skipping each feature, represented by DYMPL(-1). The performance shown here is normalized to FAPS-3D. The simulations are run with PRT record of 512 entries per memory channel. Performance of our proposed perceptron-based dynamic page mode with all the features, DYMPL, is also included for reference. As can be seen from Figure 13, features such as page utilization, page recency, page hotness, bank hit count, and bank recency have higher impact on the overall performance, since skipping any of these features from the training model results in noticeable performance drop in our

proposed scheme. On the other hand, feature like column stride has relatively lower impact on the training model, while the feature page hit count has the lowest impact because it depends exclusively on the previous hit count of the exact same page in the PRT, and cannot be generalized by the hit count of other pages. Skipping this feature results in a noticeable decrease in computation and storage overhead, while it incurs only a minor drop in overall performance (~0.3%). Similarly, column stride feature may also be skipped (by incurring a performance penalty of ~0.6%), to further save storage and computation overhead. Skipping both column stride and page hit count incurs a performance penalty of ~0.8%.

5.6.2 Record Table Size: Figure 12 shows the impact on performance by varying the size of the record tables. Performance is normalized to the FAPS-3D. The performance improvement is 4.9%, 5.5%, 5.9%, and 6.2% for PRT with 256, 512, 1024, and 2048 entries, respectively. More entries in the PRT provide deeper insight into the access behavior of the memory banks in a channel. It means more data is available for training the perceptron, which in turn help predict the page open/closure decision with better accuracy. As we double the size of PRT every time, the performance improvement starts to flatten and does not improve linearly with the table size. We have chosen PRT with 512 entries for our simulations to balance the performance gain and cost. Based on the available area and energy budget, other sizes can be chosen as well to achieve a certain performance target.

6 RELATED WORK

6.1 Dynamic Page Policies

Dynamic page policies have been studied extensively for the traditional DDRx-based main memory systems in pursuit of minimizing the row-buffer conflicts, improving memory access latency and hence, the overall system performance. Dynamic page policies are categorized into access-based policies where page open/closure decision is based on the historical memory access pattern, and time-based policies, where page is kept open for a certain time period and closed afterwards.

6.1.1 Access-Based Page Policies: Huan et al. [4] proposed an access-based page policy called the processor-directed page policy where processor keeps track of the page hits and misses and sends this information to memory controller to make future page open/closure decisions. Xu et al. [5] proposed a two-level dynamic access-based page policy where a 2-bit saturating counter is kept for each of the last n DRAM-page accesses and predicts the future page open/closure for those DRAM pages.

6.1.2 Time-Based Page Policies: A time-based page policy is proposed by Stankovic et al. [8], where the concept of live-time (interval between opening a row until last access to that row) and dead-time (interval between last access to an open row and before closing it) is used to predict the time of page closure. Kaseridis et al. [7] proposed another time-based page policy where a page is left open speculatively for tRC period, which is the minimum time between the two activations in the same bank.

6.2 Perceptron Learning in Microarchitecture

6.2.1 Branch Prediction: Perceptron learning has been used previously for branch prediction. A scheme is proposed by Jiménez et. al. [20] where instead of a traditional two-bit counter, a long branch history is used to train a perceptron to make accurate branch prediction. Once the outcome is known, weights are updated accordingly based on the correct or wrong prediction of the perceptron model.

6.2.2 Cache Reuse Prediction: Perceptron learning has also been proposed for cache reuse prediction. Jimenez et. al. [23] proposed multiperspective reuse prediction, a technique where perceptron learning is used to optimize the placement, promotion, and bypassing cache blocks and hence predict their future reuse by relying on a set of features used to train the perceptron. Teran et. al. [24] proposed the use of perceptron learning to improve the accuracy of cache reuse prediction by focusing on the addresses of previous memory instructions and various shifts of the currently accessed blocks.

6.2.3 Prefetch Prediction: Bhatia et. al. [21] used the perceptron learning based prefetch filtering, a technique used to increase the coverage of the prefetches generated by an underlaying prefetcher without negatively impacting the accuracy. Wang et. al. [22] proposed a technique that detects and traces the memory access pattern to help reject the unnecessary prefetch decisions made by the underlying stride prefetching and Markov prefetching mechanism.

7 CONCLUSION

In this paper, we have proposed a perceptron-based dynamic page policy that makes page open/closure decision by learning memory access pattern to the different DRAM pages in a bank. Our target memory platform is HBM-based main memory system; although, it can be implemented for DDRx-based main memory system as well. We have evaluated our proposed scheme over a wide selection of multi-programming SPEC CPU2006 and SPEC CPU2017 benchmark sets with varying memory intensities. Our adaptive scheme reduces main memory access latency effectively, which leads to an average performance improvement of 19.8% over the baseline close-page policy. Our scheme also out-performs an existing access-based page policy PBRM by 9.9%, and a time-based page policy INTAH by 7.6%. Our proposed scheme also performs better than another previously proposed access-based dynamic page policy FAPS-3D by 5.5%, which also targets towards massively parallel and high bandwidth 3D-stacked DRAM based main memory system.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their valuable feedback. This work is supported in part by the National Science Foundation under grants CCF-1513899. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors only and do not necessarily reflect the views of the National Science Foundation.

REFERENCES

[1] W. A. Wulf and S. A. McKee, "Hitting the memory wall," ACM SIGARCH Comput. Archit. News, vol. 23, no. 1, Mar. 1995, doi: 10.1145/216585.216588.
[2] "JEDEC Standard High Bandwidth Memory (HBM) DRAM Specification." JESD235C, Jan. 2020.

[3] M. Awasthi, D. W. Nellans, R. Balasubramonian, and A. Davis, "Prediction Based DRAM Row-Buffer Management in the Many-Core Era," Oct. 2011, doi: 10.1109/PACT.2011.31.
[4] D. Huan, Z. Li, W. Hu, and Z. Liu, "Processor Directed Dynamic Page Policy," 2006.
[5] Y. Xu, A. S. Agarwal, and B. T. Davis, "Prediction in Dynamic SDRAM Controller Policies," 2009.
[6] M. Ghasempour, A. Jaleel, J. D. Garside, and M. Luján, "HAPPY," Oct. 2016, doi: 10.1145/2989081.2989101.
[7] D. Kaseridis, J. Stuecheli, and L. K. John, "Minimalist open-page," in Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture - MICRO-44 '11, 2011, pp. 24–35, doi: 10.1145/2155620.2155624.
[8] V. V. Stankovic and N. Z. Milenkovic, "DRAM Controller with a Close-Page Predictor," 2005, doi: 10.1109/EURCON.2005.1630025.
[9] M. M. Rafique and Z. Zhu, "FAPS-3D: feedback-directed adaptive page management ccheme for 3D-stacked DRAM," Sep. 2019, doi: 10.1145/3357526.3357540.
[10] J. L. Henning, "SPEC CPU2006 benchmark descriptions," ACM SIGARCH Comput. Archit. News, vol. 34, no. 4, Sep. 2006, doi: 10.1145/1186736.1186737.
[11] "Standard performance evaluation corporation cpu2006 benchmark suite." 2006.
[12] "Standard performance evaluation corporation cpu2017 benchmark suite." 2017.
[13] D. H. Woo, N. H. Seong, D. L. Lewis, and H.-H. S. Lee, "An optimized 3D-stacked memory architecture by exploiting excessive, high-density TSV bandwidth," Jan. 2010, doi: 10.1109/HPCA.2010.5416628.
[14] J. T. Pawlowski, "Hybrid memory cube (HMC)," Aug. 2011, doi: 10.1109/HOTCHIPS.2011.7477494.
[15] M. Oskin, F. T. Chong, and T. Sherwood, "Active Pages: a computation model for intelligent memory," 1998, doi: 10.1109/ISCA.1998.694774.
[16] D. Patterson *et al.*, "A case for intelligent RAM," IEEE Micro, vol. 17, no. 2, 1997, doi: 10.1109/40.592312.
[17] Yoongu Kim, Dongsu Han, O. Mutlu, and M. Harchol-Balter, "ATLAS: A scalable and high-performance scheduling algorithm for multiple memory controllers," Jan. 2010, doi: 10.1109/HPCA.2010.5416658.
[18] Y. Kim, M. Papamichael, O. Mutlu, and M. Harchol-Balter, "Thread Cluster Memory Scheduling: Exploiting Differences in Memory Access Behavior," Dec. 2010, doi: 10.1109/MICRO.2010.51.
[19] S. I. Gallant, "Perceptron-based learning algorithms," IEEE Trans. Neural Networks, vol. 1, no. 2, Jun. 1990, doi: 10.1109/72.80230.
[20] D. A. Jimenez and C. Lin, "Dynamic branch prediction with perceptrons," 2001, doi: 10.1109/HPCA.2001.903263.
[21] E. Bhatia, G. Chacon, S. Pugsley, E. Teran, P. V. Gratz, and D. A. Jiménez, "Perceptron-based prefetch filtering," Jun. 2019, doi: 10.1145/3307650.3322207.
[22] H. Wang and Z. Luo, "Data cache prefetching with perceptron learning," 2017.
[23] D. A. Jiménez and E. Teran, "Multiperspective reuse prediction," Oct. 2017, doi: 10.1145/3123939.3123942.
[24] E. Teran, Z. Wang, and D. A. Jimenez, "Perceptron learning for reuse prediction," Oct. 2016, doi: 10.1109/MICRO.2016.7783705.
[25] N. Binkert *et al.*, "The gem5 simulator," ACM SIGARCH Comput. Archit. News, vol. 39, no. 2, May 2011, doi: 10.1145/2024716.2024718.
[26] K. Chandrasekar *et al.*, "DRAMPower: Open-source DRAM Power & Energy Estimation Tool." [Online]. Available: http://www.drampower.info.
[27] M. Jung, C. Weis, N. Wehn, M. Sadri, and L. Benini, "Optimized active and power-down mode refresh control in 3D-DRAMs," Oct. 2014, doi: 10.1109/VLSI-SoC.2014.7004159.
[28] A. Hansson, N. Agarwal, A. Kolli, T. Wenisch, and A. N. Udipi, "Simulating DRAM controllers for future system architecture exploration," Mar. 2014, doi: 10.1109/ISPASS.2014.6844484.
[29] "Micron-Technology: HBM2E." Micron-Technology, 2021, [Online]. Available: https://www.micron.com/products/ultra-bandwidth-solutions/hbm2e.
[30] "Intel Xeon Processor X5650." 2010, [Online]. Available: https://ark.intel.com/content/www/us/en/ark/products/47922/intel-xeon-processor-x5650-12m-cache-2-66-ghz-6-40-gt-s-intel-qpi.html.
[31] N. Muralimanohar, R. Balasubramonian, and N. Jouppi, "Optimizing NUCA Organizations and Wiring Alternatives for Large Caches with CACTI 6.0," Dec. 2007, doi: 10.1109/MICRO.2007.33.